DALHOUSIE UNIVERSITY

Cleaning and Transforming Data

Missing data, feature engineering, standardization, and discretization

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Missing data

What it is for Before choosing how to handle a blank, you need an assumption about why it is blank.

The rule A mechanism describes the probability that a value is unavailable. It is a property of how the data were collected and recorded, so one blank cell cannot identify it. Table 1 states what the missingness may depend on under each mechanism.

Table 1. What missingness can depend on under each mechanism.

Mechanism	What missingness can depend on
MCAR	Neither observed nor unobserved values.
MAR	Observed values; after conditioning on them, not the missing values.
MNAR	Unobserved values still matter after conditioning on observed information.

- **MCAR.** The complete cases are a random subset of the eligible records. Complete-case summaries avoid selection bias from missingness, but they are less precise.
- **MAR.** Missingness may vary with observed information. An analysis can account for that information, but MAR alone does not make an arbitrary imputation or deletion rule valid.
- **MNAR.** Dependence on unobserved values remains after conditioning on what is observed. Handling it needs external information, an explicit model, or a sensitivity analysis [3, 6].

A failure of MCAR does not mean every complete-case estimate is biased. Some correctly specified conditional models stay valid under broader conditions. Ask which quantity you are estimating and which assumption supports it.

The strategies Figure 1 lists the alternatives. The branches are choices, not steps to run in order.

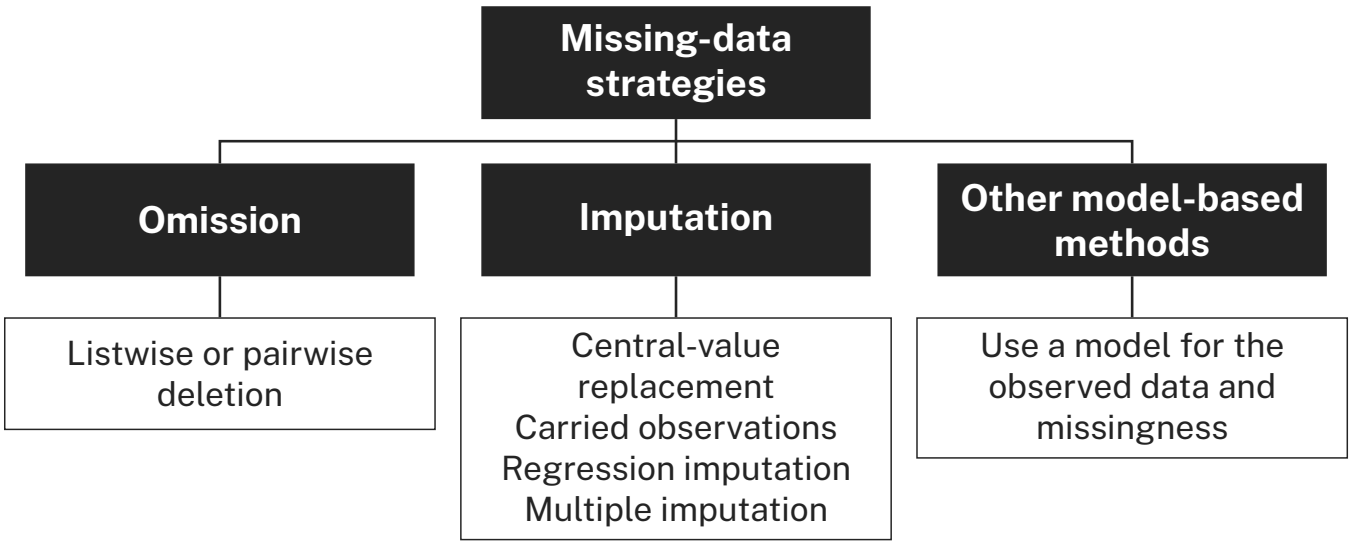


Figure 1. Alternative missing-data strategies. Branches identify alternatives, not a required sequence of operations.

- **Listwise deletion** keeps only records complete on the variables an analysis needs. **Pairwise deletion** uses the available records separately for each statistic, so a mean for one variable can use rows where another variable is missing. Report the denominator of every summary.

- **Mean, median and mode replacement** are single imputation. Filling with the observed mean leaves the observed mean unchanged, which is arithmetic and says nothing about the population mean. Adding copies of the mean adds zero deviations to the numerator of the variance while the divisor grows, so the sample variance shrinks. Median or mode replacement need not preserve the mean. None of these restores the uncertainty of the missing measurements.
- **LOCF** (last observation carried forward) repeats the previous value in a series. **BOCF** (baseline observation carried forward) repeats a designated baseline value. Backward fill uses the next observed value and is not BOCF. Sort by time and keep units, people or sites separate before any carry. A leading missing value has no earlier value to carry.
- **Regression imputation** predicts a missing value from other variables. The mean prediction alone omits residual variation. **Multiple imputation** builds several plausible completed datasets, analyses each, and pools the estimates and their uncertainty. The `mice` package implements it [1]. These model-based methods are not required for Lab 02.

Worked example A column holds 2, a missing value, 8 and 12. The observed values are 2, 8 and 12, so the observed median is 8. The code below records which row was missing, keeps the source table, and fills the missing value with 8 in a separate copy.

R

```r
raw <- data.frame(value = c(2, NA, 8, 12))
observed <- raw[!is.na(raw$value), ,
                drop = FALSE]
cleaned <- raw
centre <- median(observed$value)
cleaned$was_missing <- is.na(raw$value)
cleaned$value[cleaned$was_missing] <- centre
cleaned
# A chosen value, not a recovered
# measurement.
```

Python

```python
import pandas as pd

raw = pd.DataFrame({"value": [2, None, 8, 12]})
observed = raw.dropna(subset=["value"])
cleaned = raw.copy()
centre = observed["value"].median()
cleaned["was_missing"] = raw["value"].isna()
cleaned["value"] = (
    cleaned["value"].fillna(centre))
print(cleaned)
```

The flag and the untouched source are the evidence for the decision. The example does not justify median replacement for a monitoring dataset. In Lab 02, keeping a missing value and reporting its count is often the most defensible choice.

2 What data preparation does

What it is for Data preparation is the CRISP-DM stage that turns the table you received into the table an analysis needs. Cleaning, transforming, feature engineering, and reduction are related preparation tasks.

The rule Preparation is iterative. A summary may reveal an unexpected unit, a plot may show a recording error, and either can send you back to change an import rule. You do not have to finish every preparation step before looking at the data again. Table 2 pairs four common problems with the operation that answers each. The rest of this handout takes them one at a time.

Table 2. Data problems and possible preparation responses.

Problem	Possible response
Variables have incompatible scales	Data transformation
A useful ratio needs its own variable	Feature engineering
Records contain unavailable or invalid values	Data cleaning
Too many variables obscure an analysis	Dimensionality reduction

3 Feature engineering and units

What it is for A derived variable combines source measurements into a quantity that answers the question better than either source alone.

The rule Write the formula with its units and convert every input to those units before computing. Body mass index (BMI) is

$$\text{BMI} = \frac{\text{mass in kg}}{(\text{height in m})^2}.$$

Worked example For a mass of 82 kg and a height of 178 cm, convert the height to 1.78 m first. Then

$$\text{BMI} = \frac{82}{1.78^2} = 25.88 \text{ kg/m}^2.$$

The code applies the same formula to three people, giving 25.88, 22.41 and 29.00. BMI here illustrates arithmetic and documentation, not a clinical judgement.

R

```r
measurements <- data.frame(
  mass_kg = c(82, 61, 95),
  height_cm = c(178, 165, 181))
measurements$bmi <- measurements$mass_kg /
  (measurements$height_cm / 100)^2
round(measurements$bmi, 2)
# 25.88 22.41 29.00
```

Python

```python
import pandas as pd

measurements = pd.DataFrame({
    "mass_kg": [82, 61, 95],
    "height_cm": [178, 165, 181]})
measurements["bmi"] = (
    measurements["mass_kg"]
    / (measurements["height_cm"] / 100)**2)
print(measurements["bmi"].round(2).to_list())
# [25.88, 22.41, 29.0]
```

Both versions keep the original measurements beside the derived value. The columns come from one table, so corresponding rows describe the same person. R combines vectors by position and can recycle a shorter vector. pandas combines Series by their index labels. When the inputs come from different tables, establish the row relationship first. Equal lengths do not prove that rows correspond.

4 Reducing a dataset and sampling

What it is for Reduction makes a dataset smaller along one of three axes. Each discards something, so choose it for an analytical reason, not because the file gets smaller.

The three choices Figure 2 separates them.

- **Fewer features.** Simplifies computation or allows a two-dimensional display. Feature selection keeps some original variables. Feature projection builds new coordinates from combinations of variables. PCA is a linear projection.

- **Fewer rows.** Makes a first inspection manageable, but may omit rare conditions.
- **Fewer distinct values.** Binning turns numbers into categories and loses the detail inside each bin (Section 6).

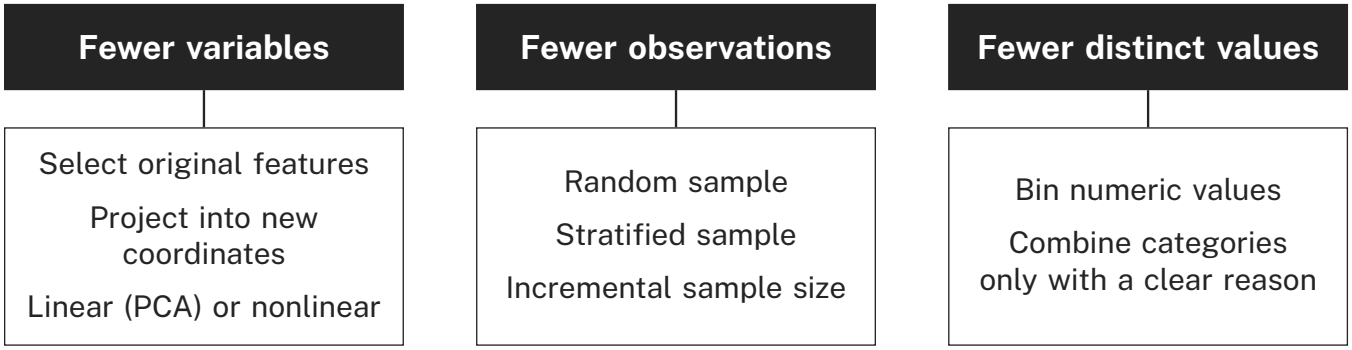


Figure 2. Three separate reduction choices.

Random and stratified samples A random sample chooses records through a defined chance mechanism. A stratified sample first splits the records into groups, then samples within each group. Stratification helps keep rare groups, and the grouping variable and allocation must follow the question. It cannot repair a biased source dataset.

Worked example Ten records come from two sites, five from each. Both samples below contain six records. The random sample may take any mix of sites. The stratified sample takes three from each site, so its site proportions (50 percent each) match the source table.

R

```r
data <- data.frame(row_id = 1:10,
  site = rep(c("A", "B"), each = 5))
set.seed(3141)
idx <- sample(seq_len(nrow(data)), size = 6)
random_sample <- data[idx, ]
# Draw three rows from each site.
idx_by_site <- split(
  seq_len(nrow(data)), data$site)
chosen <- unlist(
  lapply(idx_by_site, sample, size = 3))
stratified_sample <- data[chosen, ]
table(stratified_sample$site)
# A 3, B 3
```

Python

```python
import pandas as pd

data = pd.DataFrame({
    "row_id": range(1, 11),
    "site": ["A"]*5 + ["B"]*5})
random_sample = data.sample(
    n=6, random_state=3141)
# Draw three rows from each site.
stratified_sample = data.groupby("site").sample(
    n=3, random_state=3141)
print(stratified_sample["site"].value_counts())
# A 3, B 3
```

Differences to know

- A seed makes a draw reproducible within one software and algorithm. `set.seed(3141)` and `random_state=3141` do not give the same rows.
- Pass an integer sample size. Seventy percent of 101 rows gives 70 rows after R's integer truncation, while pandas `sample(frac=0.7)` rounds to 71. Sampling a fraction within each group can change the total again.

With proportional allocation, every record has the same chance of selection. If groups are sampled at different rates, a population estimate must weight by those probabilities. Records from the same station or period may be dependent, so randomly chosen rows are not automatically independent observations. Lab 02 does not require subsampling or a training/test split.

5 Standardization

What it is for A variable's units set its numerical spread. Methods that compare variables through distances or variance let the largest scale dominate. Standardization puts every variable on a unit-free scale.

The formula For observed values with a nonzero sample standard deviation,

$$z_i = \frac{x_i - \bar{x}}{s_x},$$

where $\bar{x}$ is the observed mean and s_x is the sample standard deviation with divisor $n - 1$. The results have no units, mean zero and sample standard deviation one.

Worked example For the vector $x = 2, 4, 4, 5, 7, 9, 12, 41$, the mean is $\bar{x} = 10.5$ and $s_x = 12.72792$. The first value becomes $z_1 = (2 - 10.5)/12.72792 = -0.6678$. Table 3 gives selected values.

Table 3. Selected standardized values, using the sample standard deviation.

x_i	$x_i - \bar{x}$	z_i
2	−8.5	−0.668
4	−6.5	−0.511
7	−3.5	−0.275
12	1.5	0.118
41	30.5	2.396

R

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
z <- scale(x)
attr(z, "scaled:center")  # 10.5
attr(z, "scaled:scale")   # 12.72792
round(z[1], 4)            # -0.6678
scale(c(x, NA))[1]        # -0.6678231
# Only the last output is NA; observed
# values remain.
```

Python

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

x = np.array([2, 4, 4, 5, 7, 9, 12, 41.])
s = StandardScaler().fit(x.reshape(-1, 1))
print(s.mean_[0])              # 10.5
print(round(s.scale_[0], 4))   # 11.9059
print(round(s.transform(x[:, None])[0, 0], 4))
# -0.7139; use ddof=1 for the R convention:
print(round(
    ((x - x.mean()) / x.std(ddof=1))[0], 4))
# -0.6678
```

Differences to know

- R's `scale()` divides by the sample standard deviation (divisor $n - 1$) [5]. scikit-learn's `StandardScaler` uses divisor n [4]. For this vector its standard deviation is 11.9059, so the first score is -0.7139 instead of -0.6678. Use `std(ddof=1)` to reproduce R.
- Both ignore missing values when estimating centre and spread, and both keep missing positions in the output.
- A constant or all-missing column needs an explicit decision. Division by zero is not a standardization. `StandardScaler` gives a constant column a scale of one, which does not give it variance one.

Standardization changes the scale, not the shape. It does not remove outliers, skewness or dependence. The value 41 is still the extreme observation, now 2.396 sample standard deviations above the mean. A one-unit change in z is a numerical comparison, not proof that two variables matter equally.

6 Discretization

What it is for Discretization replaces a number with an interval label, for a question that is naturally about categories. It discards within-bin differences and makes the result depend on the chosen boundaries, so it is not automatic cleaning.

The two rules

- **Equal width.** k bins whose boundaries are evenly spaced in measurement units, $(\max - \min)/k$ apart.
- **Equal frequency.** Boundaries at sample quantiles, so each bin holds about the same number of values. Ties make the counts approximate.

The first interval includes its lower boundary. Each later interval is open on the left and closed on the right.

Worked example Use four bins on the same vector $x = 2, 4, 4, 5, 7, 9, 12, 41$. Equal-width spacing is $(41 - 2)/4 = 9.75$, giving edges $2, 11.75, 21.5, 31.25, 41$ (Table 4). The sample quartiles give equal-frequency edges $2, 4, 6, 9.75, 41$ (Table 5).

Table 4. Equal-width bins with explicit boundaries.

Interval	Count	Members
$[2, 11.75]$	6	2,4,4,5,7,9
$(11.75, 21.5]$	1	12
$(21.5, 31.25]$	0	none
$(31.25, 41]$	1	41

Table 5. Equal-frequency bins based on sample quartiles.

Interval	Count	Members
$[2,\ 4]$	3	2, 4, 4
$(4,\ 6]$	1	5
$(6,\ 9.75]$	2	7, 9
$(9.75,\ 41]$	2	12, 41

The counts are $6, 1, 0, 1$ for equal width and $3, 1, 2, 2$ for equal frequency. Equal width keeps equal spacing between boundaries, but six of eight values share one bin. Equal frequency spreads the counts, but its last bin holds both 12 and 41 and hides their 29-unit difference.

R

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
edges <- seq(min(x), max(x), length.out = 5)
width_bins <- cut(x, breaks = edges,
                  include.lowest = TRUE)
quantiles <- quantile(
  x, probs = seq(0, 1, 0.25))
stopifnot(length(unique(quantiles)) == 5)
freq_bins <- cut(x, breaks = quantiles,
                 include.lowest = TRUE)
table(width_bins)  # 6 1 0 1
table(freq_bins)   # 3 1 2 2
```

Python

```python
import numpy as np
import pandas as pd

x = np.array([2, 4, 4, 5, 7, 9, 12, 41.])
edges = np.linspace(x.min(), x.max(), 5)
width_bins = pd.cut(
    x, bins=edges, include_lowest=True)
freq_bins = pd.qcut(x, q=4)
print(pd.Series(width_bins).value_counts(
    sort=False))
print(pd.Series(freq_bins).value_counts(
    sort=False))
```

Differences to know

- Given an integer number of bins, both `cut` functions can extend the range slightly to include the endpoints. With explicit edges, set `include.lowest = TRUE` in R or `include_lowest=True` in pandas so the minimum falls in the first bin.
- With many ties, quantile boundaries can repeat. R's `cut` rejects non-unique breaks, and `qcut` raises by default.

When boundaries repeat, use fewer intervals, choose meaningful boundaries, or keep the continuous variable. Do not split identical values across categories. Report the number of bins you obtained, not the number you asked for.

7 Representing categories

What it is for Some software needs numbers, not labels. The representation must not add information the labels do not have.

The rule Encode a nominal variable with one indicator column per category. A row has 1 in the column of its category and 0 elsewhere. Coding sites A, B and C as 1, 2 and 3 would claim an order and equal spacing that the labels do not have.

Worked example The site labels A, B, A, C give three indicator columns. The first and third rows have 1 under A, the second under B and the fourth under C.

R

```r
site <- factor(c("A", "B", "A", "C"))
indicators <- model.matrix(~ site - 1)
indicators
# Three columns, one per site; no intercept.
```

Python

```python
import pandas as pd

site = pd.Series(["A", "B", "A", "C"])
indicators = pd.get_dummies(site, dtype=int)
print(indicators)
# Three columns, one for each site.
```

Neither version assigns an order to the site names. Lab 02 needs clear variable types and documented labels. It does not need indicator encoding or a model.

8 From monitoring records to a documented table

What it is for This section combines the operations above on one monitoring record, the kind of file Lab 02 uses. The goal is an analytical table in which every change can be traced back to the raw record.

8.1 Separate the meaning of unavailable measurements

The rule Read every field as text first, list the distinct tokens, and give each one a status before any numeric conversion. A value that cannot be used in exact arithmetic may still carry information.

Worked example A concentration column is documented in mg/L. Table 6 shows five strings with four different statuses.

Table 6. A small monitoring example with distinct value-status categories.

Raw field	Status	Interpretation
`0`	measured	Exact recorded zero in the stated unit.
blank	missing	No usable measurement was recorded.
`<0.1`	censored	Below 0.1 mg/L; retain 0.1 as a limit, not a value.
`oops`	invalid	Unexpected token; investigate the original record.
`0.24`	measured	Exact recorded value, subject to provider quality flags.

- `0` and `0.24` are exact measurements. A measured zero is a value.
- `<0.1` is censored. It reports an upper limit, not a measurement of 0.1 and not a zero.
- The blank is missing under this example's documentation.
- `oops` is invalid under the numeric schema and needs investigation.

Provider flags may add restrictions, such as a provisional reading or a failed instrument check. The code below assigns the statuses by hand, keeps the raw strings and units, and checks that the raw text is unchanged.

R

```r
raw <- data.frame(
  value_text = c("0", "", "<0.1", "oops",
                 "0.24"),
  unit = rep("mg/L", 5),
  stringsAsFactors = FALSE)
cleaned <- raw
cleaned$status <- c(
  "measured", "missing", "censored",
  "invalid", "measured")
cleaned$value_mg_L <- c(0, NA, NA, NA, 0.24)
cleaned$limit_mg_L <- c(NA, NA, 0.1, NA, NA)
stopifnot(identical(
  raw$value_text, cleaned$value_text))
# These statuses follow the stated rules.
# Original strings and units remain.
```

Python

```python
import pandas as pd

raw = pd.DataFrame({
    "value_text":
        ["0", "", "<0.1", "oops", "0.24"],
    "unit": ["mg/L"]*5})
cleaned = raw.copy()
cleaned["status"] = [
    "measured", "missing", "censored",
    "invalid", "measured"]
cleaned["value_mg_L"] = [
    0, None, None, None, 0.24]
cleaned["limit_mg_L"] = [
    None, None, 0.1, None, None]
assert raw["value_text"].equals(
    cleaned["value_text"])
```

Hand assignment works for five known records. A real file needs a rule taken from the provider's data dictionary. Keep literal tokens such as `NA` as text until the documentation says whether they mean

unavailable, a category, or something else. Do not suppress conversion warnings without looking at the records that caused them.

8.2 Apply the documented rule to every row

The rule For this example only, a blank means missing, an unsigned decimal string is an exact measurement, and a less-than sign followed by such a decimal is a reporting limit. Anything else is invalid and is not silently turned into missing. The regular expressions encode these rules.

- `[0-9]+` matches one or more digits.
- `([.][0-9]+)?` allows an optional decimal part.
- The anchors `^` and `$` in R, or the full-match operation in Python, require the whole string to match.

Worked example Applied to the five strings, the rule reproduces the statuses of Table 6. Exact values go to `value_mg_L`, the limit 0.1 goes to a separate `limit_mg_L` column, and the exact values are converted with $1\ \mathrm{mg/L} = 1000\ \mu\mathrm{g/L}$, so 0.24 mg/L becomes 240 µg/L.

R

```r
raw <- data.frame(
  value_text = c("0", "", "<0.1", "oops",
                 "0.24"),
  unit = rep("mg/L", 5))
cleaned <- raw
text <- trimws(raw$value_text)
# This provider defines blank as missing.
missing <- text == ""
censored <- grepl("^<[0-9]+([.][0-9]+)?$", text)
measured <- grepl("^[0-9]+([.][0-9]+)?$", text)
cleaned$status <- ifelse(missing, "missing",
  ifelse(censored, "censored",
    ifelse(measured, "measured", "invalid")))
cleaned$value_mg_L <- NA_real_
cleaned$value_mg_L[measured] <-
  as.numeric(text[measured])
cleaned$limit_mg_L <- NA_real_
cleaned$limit_mg_L[censored] <-
  as.numeric(sub("^<", "", text[censored]))
cleaned$value_ug_L <- 1000 * cleaned$value_mg_L
cleaned
```

Python

```python
import pandas as pd
raw = pd.DataFrame({
    "value_text":
        ["0", "", "<0.1", "oops", "0.24"],
    "unit": ["mg/L"]*5})
cleaned = raw.copy()
text = raw["value_text"].str.strip()
# This provider defines blank as missing.
missing = text.eq("")
censored = text.str.fullmatch(
    r"<[0-9]+([.][0-9]+)?")
measured = text.str.fullmatch(
    r"[0-9]+([.][0-9]+)?")
cleaned["status"] = "invalid"
for mask, label in [(missing, "missing"),
                    (censored, "censored"),
                    (measured, "measured")]:
    cleaned.loc[mask, "status"] = label
cleaned["value_mg_L"] = pd.to_numeric(
    text.where(measured))
cleaned["limit_mg_L"] = pd.to_numeric(
    text.where(censored).str.removeprefix("<"))
cleaned["value_ug_L"] = (
    1000 * cleaned["value_mg_L"])
print(cleaned)
```

These rules are deliberately narrow. Real records may use signs, scientific notation, other missing codes, or other limit formats, so inspect the tokens before adapting them. The documentation and the token audit decide the classification, and the code makes it reproducible across all rows. The cleaned copy never replaces a censored result with zero or half the limit. That would need an extra assumption and a sensitivity analysis, and Helsel shows how such substitution distorts later estimates [2].

8.3 Check units, keys and counts

The rule Keep the downloaded file unchanged. Before combining observations, identify the station, time, parameter, unit and quality-flag fields, and state the observational unit. Many tables hold one row per station, time and parameter, not one row per station.

Duplicates The observational unit defines a duplicate. Choose the key columns, for example station, date and parameter, and count rows that share a key before removing anything.

- R. `duplicated(d[key]) | duplicated(d[key], fromLast = TRUE)` marks every row in a repeated group.
- Python. `d.duplicated(subset=key, keep=False)` does the same.

Worked example Four nitrate rows come from two stations. Station S1 has two rows on 2026-09-01, so the key station, date and parameter repeats. Both rows are marked, and neither is removed. The two values, 0.24 and 0.31 mg/L, differ, so the repeat is not an exact double export.

R

```r
d <- data.frame(
  station = c("S1", "S1", "S1", "S2"),
  date = c("2026-09-01", "2026-09-01",
           "2026-09-02", "2026-09-01"),
  parameter = rep("nitrate", 4),
  value_mg_L = c(0.24, 0.31, 0.22, 0.18)
)
key <- c("station", "date", "parameter")
repeated <- duplicated(d[key]) |
  duplicated(d[key], fromLast = TRUE)
sum(repeated)   # 2
d[repeated, ]
#   station       date parameter value_mg_L
# 1      S1 2026-09-01   nitrate       0.24
# 2      S1 2026-09-01   nitrate       0.31
```

Python

```python
import pandas as pd

d = pd.DataFrame({
    "station": ["S1", "S1", "S1", "S2"],
    "date": ["2026-09-01", "2026-09-01",
             "2026-09-02", "2026-09-01"],
    "parameter": ["nitrate"] * 4,
    "value_mg_L": [0.24, 0.31, 0.22, 0.18],
})
key = ["station", "date", "parameter"]
repeated = d.duplicated(subset=key, keep=False)
print(repeated.sum())   # 2
print(d[repeated])
#   station        date parameter  value_mg_L
# 0      S1  2026-09-01   nitrate        0.24
# 1      S1  2026-09-01   nitrate        0.31
```

A repeated key is a question for the documentation. Two rows may be an accidental double export, or they may differ in depth, sample fraction or laboratory replicate, which means the key was incomplete.

Units Convert only with a named rule, such as $1\,\mathrm{mg/L} = 1000\,\mu\mathrm{g/L}$. Keep the original unit field and put the converted value in a new, clearly named column. Do not combine concentrations of different parameters because they share a unit. Flagging an unusual reading is a reason to investigate, not evidence that it should be deleted.

Counts Report the number of exact, missing, censored, invalid and excluded records. A mean of the exact measurements describes that subset. When censoring or exclusions are substantial it may not represent all sampled conditions, so state the denominator and the limitation beside the summary.

8.4 The Lab 02 workflow

Figure 3 and the list below give the order of work for one monitoring record.

1. Read one approved monitoring record and identify its observational unit.
2. Inspect types, units, missing tokens, reporting limits and quality flags.
3. Create a cleaned copy and record a reason for every changed or excluded value.
4. Compare summaries before and after preparation while keeping the raw record.
5. If categories help the question, compare two binning rules on one suitable numeric variable.

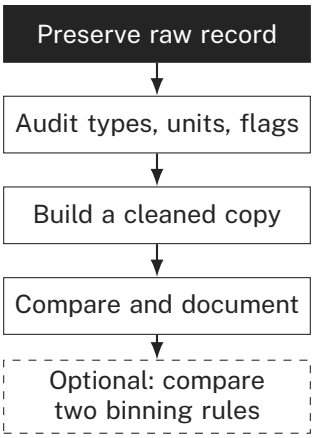


Figure 3. A preparation workflow that keeps raw evidence separate from the analytical copy.

No PCA, model fitting or training/test split is required for Lab 02. Use one approved language. Follow the syllabus for the private submission and later portfolio release, and preserve the same submitted commit and its recorded history.

9 Summary

- A missing-data mechanism is an assumption about the recording process. Any replacement goes in a flagged copy, never over the source.
- A derived variable is only as correct as its units. Convert before you compute.
- Standardizing changes scale, not shape. State the divisor, because R and scikit-learn differ.
- Equal-width and equal-frequency bins answer different questions, and both lose within-bin detail.
- Indicators encode membership without inventing an order.
- In a monitoring record, measured, missing, censored and invalid are different statuses. Keep the raw string, store limits apart from values, and report every count.

References

[1] S. van Buuren and K. Groothuis-Oudshoorn. “mice: multivariate imputation by chained equations in R”. In: *Journal of Statistical Software* 45.3 (2011), pp. 1–67.

[2] D. R. Helsel. “Fabricating data: how substituting values for nondetects can ruin results, and what can be done about it”. In: *Chemosphere* 65.11 (2006), pp. 2434–2439.

[3] R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. 3rd ed. Wiley, 2019.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. “Scikit-learn: machine learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[5] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2026.

[6] D. B. Rubin. “Inference and missing data”. In: *Biometrika* 63.3 (1976), pp. 581–592.